

Q1.a

No.	Time	Source	Destination	Protocol	Length	Info
1773	2025-04-08 21:20:35.895680	172.19.111.130	50.19.22.197	HTTP	538	GET /image/webp HTTP/1.1
2214	2025-04-08 21:20:37.324093	50.19.22.197	172.19.111.130	HTTP	402	HTTP/1.1 200 OK (image/webp)
2216	2025-04-08 21:20:37.336441	172.19.111.130	50.19.22.197	HTTP	479	GET /favicon.ico HTTP/1.1
2335	2025-04-08 21:20:39.137802	50.19.22.197	172.19.111.130	HTTP	287	HTTP/1.1 404 NOT FOUND (text/html)

URL: <http://httpbin.org/image/webp>

Method: GET

Use any other method to perform the same action: NO

Q1.b

Ethernet II, Src: Intel_c5:66:bf (04:ec:d8:c5:66:bf), Dst: RuijieNetwor_e6:af:c9 (Internet Protocol Version 4, Src: 172.19.111.130, Dst: 50.19.22.197

IP address: 172.19.111.130

MAC address: 04-EC-D8-C5-66-BF

The purposes of IP address and MAC address:

- IP address is used to locate devices in a larger network (such as the Internet), allowing data packets to be transmitted between different networks.
- MAC address is used to locate devices within the same local area network (LAN), ensuring that data frames can be sent directly to the correct physical device.

ARP protocol: Address Resolution Protocol (ARP) is used to resolve IP addresses to MAC addresses. When a device needs to send data to another device within the same local area network, it uses ARP to find the MAC address of the target device. ARP sends an ARP request to all devices within the local area network in a broadcast manner, and the target device responds with its MAC address. The sending device can then use this MAC address to construct an Ethernet frame.

Q1.c

No.	Time	Source	Destination	Protocol	Length	Info
18	2025-04-08 21:20:28.007081	172.19.111.130	50.19.22.197	TCP	66	63468 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
24	2025-04-08 21:20:28.240619	50.19.22.197	172.19.111.130	TCP	66	80 → 63468 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1460 SACK_PERM WS=
25	2025-04-08 21:20:28.240748	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0
1773	2025-04-08 21:20:35.895680	172.19.111.130	50.19.22.197	HTTP	538	GET /image/webp HTTP/1.1
2028	2025-04-08 21:20:36.129600	50.19.22.197	172.19.111.130	TCP	60	80 → 63468 [ACK] Seq=1 Ack=485 Win=28160 Len=0
2209	2025-04-08 21:20:37.312374	50.19.22.197	172.19.111.130	TCP	280	80 → 63468 [PSH, ACK] Seq=1 Ack=485 Win=28160 Len=226 [TCP PDU reassembl
2211	2025-04-08 21:20:37.323719	50.19.22.197	172.19.111.130	TCP	1514	80 → 63468 [ACK] Seq=227 Ack=485 Win=28160 Len=1460 [TCP PDU reassembled
2212	2025-04-08 21:20:37.323774	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=485 Ack=1687 Win=131328 Len=0
2213	2025-04-08 21:20:37.324093	50.19.22.197	172.19.111.130	TCP	8814	80 → 63468 [ACK] Seq=1687 Ack=485 Win=28160 Len=8760 [TCP PDU reassembl
2214	2025-04-08 21:20:37.324093	50.19.22.197	172.19.111.130	HTTP	402	HTTP/1.1 200 OK (image/webp)
2215	2025-04-08 21:20:37.324121	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=485 Ack=10795 Win=131328 Len=0
2216	2025-04-08 21:20:37.336441	172.19.111.130	50.19.22.197	HTTP	479	GET /favicon.ico HTTP/1.1

TCP handshakes occur on the following lines:

Line 18: The client (172.19.111.130) sends a SYN request to the server (50.19.22.197).

Line 24: Server (50.19.22.197) sends SYN-ACK response to client (172.19.111.130).

Line 25: The client (172.19.111.130) sends an ACK acknowledgement to server(50.19.22.197).

Explain the components of a TCP handshake in detail:

- SYN: Synchronize Sequence Numbers synchronizes the initial sequence numbers of two nodes when a connection is established.
- ACK: Acknowledgment, used to acknowledge received packets.

- Sequence Numbers: Serial number, used to identify the order of sent packets.
- Acknowledgment Numbers: indicates the acknowledgement number that identifies the next byte to be received.
- Win: Window size, used for flow control, represents the number of bytes that the receiver can receive.
- Len: length, indicating the length of the TCP segment.
- IP address: Source IP address and destination IP address, which identify the two nodes for communication.
- Port: Source port and destination port, which identify the application for communication.

Each handshake in detail:

tcp.stream eq 4

No.	Time	Source	Destination	Protocol	Length	Info
18	2025-04-08 21:20:28.007081	172.19.111.130	50.19.22.197	TCP	66	63468 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
24	2025-04-08 21:20:28.240619	50.19.22.197	172.19.111.130	TCP	66	80 → 63468 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1460 SACK_PERM WS=
25	2025-04-08 21:20:28.240748	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0

[Stream index: 4]
 Transmission Control Protocol, Src Port: 63468, Dst Port: 80, Seq: 0, Len: 0
 Source Port: 63468
 Destination Port: 80
 [Stream index: 4]
 [Stream Packet Number: 1]
 > [Conversation completeness: Incomplete, DATA (15)]
 [TCP Segment Len: 0]
 Sequence Number: 0 (relative sequence number)
 Sequence Number (raw): 1901687305
 [Next Sequence Number: 1 (relative sequence number)]
 Acknowledgment Number: 0
 Acknowledgment number (raw): 0
 1000 = Header Length: 32 bytes (8)
 > Flags: 0x002 (SYN)
 Window: 64240

First handshake(SYN):

the number of segments involved:1

flags:0x002

raw sequence numbers:1901687305

raw acknowledgment numbers:0

ports: Src Port: 63468, Dst Port: 80

IP addresses: Src: 172.19.111.130, Dst: 50.19.22.197

tcp.stream eq 4

No.	Time	Source	Destination	Protocol	Length	Info
18	2025-04-08 21:20:28.007081	172.19.111.130	50.19.22.197	TCP	66	63468 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
24	2025-04-08 21:20:28.240619	50.19.22.197	172.19.111.130	TCP	66	80 → 63468 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1460 SACK_PERM
25	2025-04-08 21:20:28.240748	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0

[Stream index: 4]
 Transmission Control Protocol, Src Port: 80, Dst Port: 63468, Seq: 0, Ack: 1, Len: 0
 Source Port: 80
 Destination Port: 63468
 [Stream index: 4]
 [Stream Packet Number: 2]
 > [Conversation completeness: Incomplete, DATA (15)]
 [TCP Segment Len: 0]
 Sequence Number: 0 (relative sequence number)
 Sequence Number (raw): 1979500465
 [Next Sequence Number: 1 (relative sequence number)]
 Acknowledgment Number: 1 (relative ack number)
 Acknowledgment number (raw): 1901687306
 1000 = Header Length: 32 bytes (8)
 > Flags: 0x012 (SYN, ACK)

Second handshake(SYN,ACK):

the number of segments involved:1

flags:0x012

raw sequence numbers:1979500465

raw acknowledgment numbers:1901687306

ports: Src Port: 80, Dst Port: 63468

IP addresses: Src: 50.19.22.197, Dst: 172.19.111.130

The image shows a Wireshark packet capture of a TCP connection. The packet list on the left shows a series of packets between 50.19.22.197 and 172.19.111.130. Packet 2282 is highlighted, showing a TCP segment with Seq=1, Ack=1, Win=131328, Len=0. The packet details pane on the right shows the following information:

- Transmission Control Protocol, Src Port: 63468, Dst Port: 80, Seq: 1, Ack: 1, Len: 0
- Source Port: 63468
- Destination Port: 80
- [Stream index: 4]
- [Stream Packet Number: 3]
- > [Conversation completeness: Incomplete, DATA (15)]
- [TCP Segment Len: 0]
- Sequence Number: 1 (relative sequence number)
- Sequence Number (raw): 1901687306
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 1979500466
- 0101 = Header Length: 20 bytes (5)
- > Flags: 0x010 (ACK)
- Window: 513
- [Calculated window size: 131328]

Third handshake(ACK):

the number of segments involved:1

flags:0x010

raw sequence numbers:1901687306

raw acknowledgment numbers:1979500466

ports: Src Port: 63468, Dst Port: 80

IP addresses: Src: 172.19.111.130, Dst: 50.19.22.197

The image shows a Wireshark packet capture of a TCP connection. The packet list on the left shows a series of packets between 172.19.111.130 and 50.19.22.197. Packet 2283 is highlighted, showing a TCP segment with Seq=1, Ack=1, Win=131328, Len=0. The packet details pane on the right shows the following information:

- Flags: 0x010 (ACK)
- Window: 513
- [Calculated window size: 131328]
- [Window size scaling factor: 256]
- Checksum: 0x6488 [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- > [Timestamps]
- [Time since first frame in this TCP stream: 0.233667000 seconds]
- [Time since previous frame in this TCP stream: 0.000129000 seconds]
- > [SEQ/ACK analysis]

RTT for the TCP handshake:0.233667s

During the TCP handshake, the following relevant parameters are exchanged:

- **Maximum Segment Size (MSS):** It defines the maximum amount of data that can be carried in a single TCP segment. **First handshake:1460; Second handshake:1460**

- **Receiver Window Size:** This parameter, represented as "Win" in the capture, indicates the amount of buffer space available at the receiver. It's used for flow control, allowing the sender to adjust the amount of data it sends based on the receiver's capacity. **First handshake: 64240; Second handshake:26883; Third handshake: 131328**

- **Window Scale (WS):** This is used to scale the window size value. It helps in expanding the window size beyond the default maximum, enabling more efficient data transfer when larger buffers are available. **First handshake: 256; Second handshake: 256**

Q1.d

No.	Time	Source	Destination	Protocol	Length	Info
18	2025-04-08 21:20:28.007081	172.19.111.130	50.19.22.197	TCP	66	63468 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
24	2025-04-08 21:20:28.240619	50.19.22.197	172.19.111.130	TCP	66	80 → 63468 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1460 SACK_PERM WS=
25	2025-04-08 21:20:28.240748	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0
1773	2025-04-08 21:20:35.895680	172.19.111.130	50.19.22.197	HTTP	538	GET /image/webp HTTP/1.1
2028	2025-04-08 21:20:36.129600	50.19.22.197	172.19.111.130	TCP	60	80 → 63468 [ACK] Seq=1 Ack=485 Win=28160 Len=0
2209	2025-04-08 21:20:37.312374	50.19.22.197	172.19.111.130	TCP	280	80 → 63468 [PSH, ACK] Seq=1 Ack=485 Win=28160 Len=226 [TCP PDU reassembl
2211	2025-04-08 21:20:37.323719	50.19.22.197	172.19.111.130	TCP	1514	80 → 63468 [ACK] Seq=227 Ack=485 Win=28160 Len=1460 [TCP PDU reassembled
2212	2025-04-08 21:20:37.323774	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=485 Ack=1687 Win=131328 Len=0
2213	2025-04-08 21:20:37.324093	50.19.22.197	172.19.111.130	TCP	8814	80 → 63468 [ACK] Seq=1687 Ack=485 Win=28160 Len=8760 [TCP PDU reassemble
2214	2025-04-08 21:20:37.324093	50.19.22.197	172.19.111.130	HTTP	402	HTTP/1.1 200 OK (image/webp)
2215	2025-04-08 21:20:37.324121	172.19.111.130	50.19.22.197	TCP	54	63468 → 80 [ACK] Seq=485 Ack=10795 Win=131328 Len=0
2216	2025-04-08 21:20:37.336441	172.19.111.130	50.19.22.197	HTTP	479	GET /favicon.ico HTTP/1.1

-**Number of segments involved:** 5

-**Flags:** For example, SYN (used for establishing a connection), ACK (used for confirmation),
First segment: ACK; Last segment: ACK

-**Raw sequence numbers:** When the client initiates a connection request, it randomly selects an initial sequence number (ISN), say x for example. When sending the message segment containing HTTP request data, the sequence number of the first byte data is x + 1 (the SYN flag occupies one sequence number). Subsequently, the sequence number of each subsequent byte of data will increase sequentially. When the server responds with data, it also has its own initial sequence number, which increases in accordance with the byte order.

First segment:1979500466

```
[TCP Segment Len: 0]
Sequence Number: 1      (relative sequence number)
Sequence Number (raw): 1979500466
```

Last segment: 1979502152

```
[TCP Segment Len: 8760]
Sequence Number: 1687   (relative sequence number)
Sequence Number (raw): 1979502152
```

-**Raw acknowledgment numbers:** After receiving the message segment from the sender, the recipient will set a confirmation number in the reply message. For example, if the client sends data starting from sequence number n and with a length of l, and the server receives it correctly, it will set the confirmation number in the reply message to n + l, indicating the sequence number of the next byte to be received, which is to inform the sender which data has been successfully received.

First segment:1901687790;

```
[Next Sequence Number: 1      (relative sequence number)]
Acknowledgment Number: 485     (relative ack number)
Acknowledgment number (raw): 1901687790
```

Last segment: 1901687790

```
[Next Sequence Number: 10447   (relative
Acknowledgment Number: 485     (relative a
Acknowledgment number (raw): 1901687790
```

-**Ports:** Source Port: When the client initiates an HTTP request, it randomly selects a port from the range of available ports (typically 1024 - 65535) as the source port. It is used to identify which process on the client initiated the request.

Destination Port: For a regular HTTP request, the destination port is usually 80; for an HTTPS request, it is 443. These ports are used to let network devices and servers know that the request is an HTTP(S) request, so that corresponding processing can be carried out.

First segment: Src Port: 80, Dst Port: 63468

Last segment: Src Port: 80, Dst Port: 63468

-**Source IP Address:** This refers to the IP address of the client device that initiates the HTTP request. This address identifies the source of the request.

-**Destination IP Address:** This is the IP address of the server providing HTTP services. The client sends the request to the corresponding server through this address.

First segment:

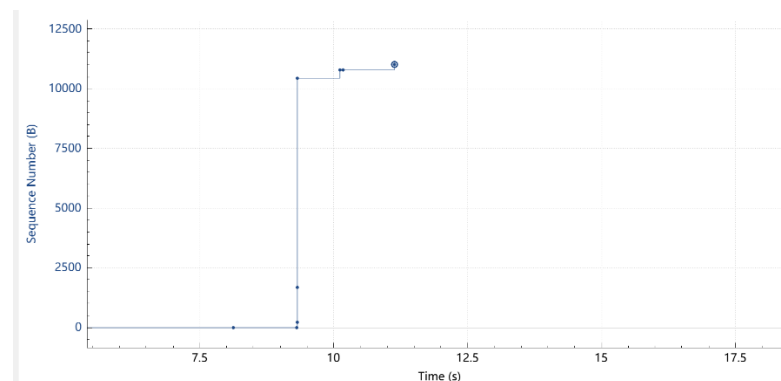
Src: 50.19.22.197, Dst: 172.19.111.130

Last segment:

Src: 50.19.22.197, Dst: 172.19.111.130

TCP segments needed to transfer the file: 5

TCP Stream Time-Sequence Graph:



Data point analysis:

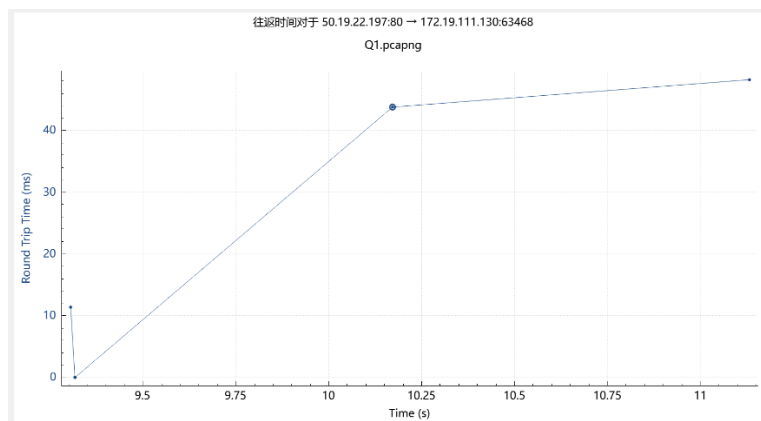
- Initial phase (0 - about 8.5s) : The ordinate sequence number is always close to 0 in the interval from 0 to about 8.5s, indicating that the serial number has hardly changed during this period, and is in a relatively stable state, with no obvious data transmission or associated serial number growth.

- Abrupt phase (about 8.5-9.5s) : When the horizontal coordinate is about 8.5s, the ordinate sequence number suddenly jumps from close to 0 to about 10000, there is a sharp increase. This means that a large number of data transfers occurred at that point in time or there was a large serial number update event, possibly the beginning of a large data sending or receiving operation.

- Plateau stage (about 9.5-11s) : From about 9.5s to 11s in the horizontal coordinate, the ordinate serial number remains unchanged at about 10000, indicating that during this period of time, data transmission is relatively stable, and no new large amount of data causes the serial number to change, which may be due to the continuous transmission of a fixed size data block or the waiting gap in data transmission.

- Small increase stage (after about 11s) : After the horizontal coordinate exceeds 11s, the serial number has a small increase to a position slightly above 10000, but the increase is much smaller than the previous mutation stage. This may indicate a small amount of new data transfer, but it is smaller than the previous large data transfer.

TCP RTT graph:



Data trend analysis

- Initial stage (9.5-10s) : In the interval from 9.5s to 10s, the round-trip time rose rapidly from close to 0ms to about 40ms, showing an obvious linear growth trend. This indicates that during this time, the latency of the network connection is increasing rapidly, possibly because the connection is starting to be established or a large amount of data is starting to be transmitted, resulting in a longer network response time.
- Steady growth stage (10-11s) : From 10s to 11s, the round trip time slowly increases from about 40ms to slightly higher than 40ms, and the growth trend is relatively flat. Note During this period, the network connection has been established, but there is still a certain delay growth. It may be that there is continuous data transmission on the network, or the network congestion is slowly increasing.

Possible network assumptions:

- Network connection establishment: The rapid growth in the initial phase, most likely when the network connection was established, went through a process such as a three-way handshake, resulting in a rapid rise in latency.
- Network congestion or data transmission: The subsequent steady growth may be because after the connection is established, there is continuous data transmission, and the data traffic in the network gradually increases, resulting in a certain degree of network congestion, and then the round-trip time continues to increase. It is also possible that there are some unstable factors in the link, causing the delay to gradually become larger.

Q1.e

1772	2025-04-08	21:20:35.894478	172.19.111.130	172.20.255.104	DNS	71 Standard query 0x1c67 A httpbin.org
1777	2025-04-08	21:20:35.903831	172.20.255.104	172.19.111.130	DNS	87 Standard query response 0x1c67 A httpbin.org A 50.19.22.197
1783	2025-04-08	21:20:35.911343	172.19.111.130	172.20.255.104	DNS	94 Standard query 0x5b1b A nav-edge.smartscreen.microsoft.com
1784	2025-04-08	21:20:35.911492	172.19.111.130	172.20.255.104	DNS	94 Standard query 0x0026 HTTPS nav-edge.smartscreen.microsoft.com
1785	2025-04-08	21:20:35.913778	172.20.255.104	172.19.111.130	DNS	214 Standard query response 0x5b1b A nav-edge.smartscreen.microsoft.com CNAME
1786	2025-04-08	21:20:35.913944	172.20.255.104	172.19.111.130	DNS	258 Standard query response 0x0026 HTTPS nav-edge.smartscreen.microsoft.com
2355	2025-04-08	21:20:40.604746	172.19.111.130	172.20.255.104	DNS	82 Standard query 0x2716 A h-adashx.ut.taobao.com

1. DNS Query Segment (Record 1772)

- Source IP Address: `172.19.111.130`, indicating the IP of the device that initiates the DNS query request.
- Destination IP Address: `172.20.255.104`, which is the IP address of the DNS server to receive the query request.
- Protocol: DNS (Domain Name System).
- Transaction ID: `0x1c67`, used to match the query and the response. This value remains the

same in the query and the corresponding response.

- Query Type: `A` record query. An A record, short for Address Record, is used to map a domain name to an IPv4 address.
- Query Domain Name: `httpbin.org`, that is, the target domain name for which the IP address is requested to be resolved.

2. DNS Response Segment (Record 1777)

- Source IP Address: `172.20.255.104`, the IP of the DNS server as the sender of the response.
- Destination IP Address: `172.19.111.130`, the IP of the client receiving the response.
- Protocol: DNS.
- Transaction ID: `0x1c67`, matching the transaction ID of the query message to confirm that the response corresponds to the query.
- Response Code: The abnormal response code is not clearly shown here (if the resolution is normal, it is generally 0, indicating no error).
- Answer Section: Contains the IPv4 address `50.19.22.197` corresponding to the query domain name `httpbin.org`, completing the mapping resolution from the domain name to the IP address.

Q1.f

```
C:\Users\ASUS>nslookup httpbin.org
Server:  Unknown
Address:  172.20.255.104

Non-authoritative answer:
Name:     httpbin.org
Addresses: 50.19.22.197
           54.227.146.83
           54.152.142.77
           34.238.6.191

C:\Users\ASUS>nslookup -type=NS httpbin.org
Server:  Unknown
Address:  172.20.255.104

Non-authoritative answer:
httpbin.org    nameserver = ns-173.awsdns-21.com
httpbin.org    nameserver = ns-1555.awsdns-02.co.uk
httpbin.org    nameserver = ns-1053.awsdns-03.org
httpbin.org    nameserver = ns-884.awsdns-46.net
```

1. IP Addresses of `httpbin.org`

From the output of `nslookup httpbin.org`, the corresponding IPv4 addresses of `httpbin.org` are: `50.19.22.197`, `54.227.146.83`, `54.152.142.77`, `34.238.6.191`. This indicates that the domain name may be configured with multiple IP addresses, which could be for purposes such as load balancing or high - availability.

2. Authoritative Name Servers of `httpbin.org`

By executing `nslookup -type=NS httpbin.org`, the following authoritative name servers are obtained:

- `ns-173.awsdns-21.com`
- `ns-1555.awsdns-02.co.uk`
- `ns-1053.awsdns-03.org`
- `ns-884.awsdns-46.net`

These name servers are responsible for maintaining the DNS records of `httpbin.org` and providing authoritative responses to resolution requests for this domain name.

3. Other Relevant Information

- Non - authoritative Answer: The output repeatedly mentions "Non - authoritative answer", indicating that the query results do not come directly from the authoritative name servers, but rather from local DNS caches or intermediate nodes in the recursive query process.
- DNS Server Information: The displayed DNS server address is `172.20.255.104` with the name "Unknown", suggesting that the identity of the DNS server used for the query has not been clearly identified. It is likely the DNS server in the local network with default configuration.

Q1.g

1. Comparison of IP Addresses

- Wireshark Capture (Point e): In the previous Wireshark packet capture analysis, for the DNS resolution of `httpbin.org`, the obtained IP address was `50.19.22.197`.
 - Output of `nslookup` Tool (Point f): Using the `nslookup` tool, the IP addresses of `httpbin.org` obtained are `54.152.142.77`, `50.19.22.197`, `34.238.6.191`, `54.227.146.83`.
- It can be seen that the IP address obtained from the Wireshark capture is one of the multiple IP addresses obtained by the `nslookup` tool. This is because `nslookup` may retrieve all the available IP addresses configured for the domain name, while the Wireshark capture only records one IP address returned during a specific DNS resolution process.

2. Comparison of Authoritative Name Servers

- Wireshark Capture (Point e): The information about the authoritative name servers of `httpbin.org` was not clearly shown.
- Output of `nslookup` Tool (Point f): The authoritative name servers obtained are `ns-1555.awsdns-02.co.uk`, `ns-1053.awsdns-03.org`, `ns-884.awsdns-46.net`, `ns-173.awsdns-21.com`.

That is, the Wireshark capture did not involve information related to authoritative name servers, while the `nslookup` tool can specifically query and display this kind of information.

Q1.h.1

1486	2025-04-08 21:20:34.607813	172.19.111.130	172.19.111.130	ICMP	120 Destination unreachable (Port unreachable)
1679	2025-04-08 21:20:35.269175	172.19.111.130	224.0.0.251	MDNS	70 Standard query 0x0000 A wpad.local, "QM" question
1680	2025-04-08 21:20:35.269619	172.19.111.130	224.0.0.251	MDNS	70 Standard query 0x0000 AAAA wpad.local, "QM" question
1681	2025-04-08 21:20:35.270034	fe80::6cb0:6da3:f5a... ff02::fb		MDNS	90 Standard query 0x0000 AAAA wpad.local, "QM" question
1682	2025-04-08 21:20:35.270443	fe80::6cb0:6da3:f5a... ff02::fb		MDNS	90 Standard query 0x0000 A wpad.local, "QM" question
1708	2025-04-08 21:20:35.445698	172.19.111.130	172.19.127.255	NBNS	92 Name query NB WPAD<00>
1709	2025-04-08 21:20:35.454251	172.19.127.254	172.19.111.130	ICMP	120 Destination unreachable (Port unreachable)
1772	2025-04-08 21:20:35.894478	172.19.111.130	172.20.255.104	DNS	71 Standard query 0x1c67 A httpbin.org
1777	2025-04-08 21:20:35.903831	172.20.255.104	172.19.111.130	DNS	87 Standard query response 0x1c67 A httpbin.org A 50.19.22.197
1783	2025-04-08 21:20:35.911343	172.19.111.130	172.20.255.104	DNS	94 Standard query 0x5b1b A nav-edge.smartscreen.microsoft.com
1784	2025-04-08 21:20:35.911400	172.19.111.130	172.20.255.104	DNS	94 Standard query 0x0000 HTTP nav-edge.smartscreen.microsoft.com

✓ User Datagram Protocol, Src Port: 5353, Dst Port: 5353

Source Port: 5353
Destination Port: 5353
Length: 36
Checksum: 0xc490 [unverified]
[Checksum Status: Unverified]
[Stream index: 39]
[Stream Packet Number: 3]
> [Timestamps]
UDP payload (28 bytes)

Internet Protocol Version 4, Src: 172.19.111.130, Dst: 224.0.0.251

0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 56
Identification: 0xac59 (44121)
> 000. = Flags: 0x0
...0 0000 0000 0000 = Fragment Offset: 0
> Time to Live: 1
Protocol: UDP (17)
Header Checksum: 0x0000 [validation disabled]
[Header checksum status: Unverified]
Source Address: 172.19.111.130
Destination Address: 224.0.0.251
[Stream index: 25]

- Number of segments involved: The UDP protocol is connectionless, unlike TCP which divides data into segments for transmission due to large data volume. Usually, a UDP packet carries relatively small amounts of data and transmits independently.
- Flags: The UDP protocol header does not have flags such as SYN, ACK, FIN, etc. for connection management and data transmission control functions as in the TCP protocol. Therefore, when analyzing this UDP packet segment, there is no concept of related flags.
- Raw sequence numbers and raw acknowledgment numbers: These concepts mainly exist in the TCP protocol to ensure ordered data transmission and confirm received data. The UDP protocol does not guarantee ordered arrival of data and reliable transmission, so there is no mechanism for raw sequence numbers and raw acknowledgment numbers, and thus no related values.
- Source Port: The value is 5353. The source port is used to identify the application or process that sends this UDP message. The receiving party can return the data to the corresponding process based on this port when replying.
- Destination Port: The value is 5353. The destination port indicates the target application or process to which this UDP message is sent. The receiving device will deliver the message to the corresponding process for processing based on this port.
- Source IP Address: `172.19.111.130`.
- Destination IP Address: `224.0.0.251`, which is a multicast address. This means that the UDP packet is not sent to a single specific device, but to a group of receiving devices that are interested in this multicast address and have made relevant configurations.

Q1.h.2

1. Application - layer Protocol and Its Purpose

- Application - layer Protocol: It may be mDNS (Multicast DNS).
- Purpose: mDNS is used for name resolution among devices within small - scale networks (such as home local area networks) without traditional DNS servers. For example, a newly - connected smart speaker in a network can use mDNS to enable devices like mobile phones on the same local area network to recognize its name, facilitating subsequent connection and control. It enables device discovery and communication without complex network configurations.

2. Reasons for Choosing UDP Instead of TCP

- UDP Characteristics Meet Requirements: UDP is connectionless, eliminating the overhead of connection establishment such as the TCP three - way handshake, resulting in fast transmission. mDNS mainly exchanges name resolution information quickly within local networks and has high requirements for real - time performance. A small number of packet losses do not affect the overall function. For instance, if a device occasionally fails to receive a resolution request, it can try again. However, the reliable transmission mechanisms of TCP, such as acknowledgment and re - transmission, will bring additional latency and resource consumption, making it unsuitable for scenarios that emphasize quick response.

3. Whether the Protocol Can Use TCP Connections and the Reasons

- Theoretical and Practical Situations: Theoretically, mDNS can use TCP connections, but in practice, it is rarely adopted.
- Reasons: TCP connection management is complex and requires a significant amount of system resources for connection establishment, maintenance, and termination. The design of mDNS aims for simple and efficient name resolution in local networks. Using TCP would add unnecessary complexity and latency, going against its design. Moreover, when the local network environment is relatively stable, the unreliability of UDP has a minimal impact, which is sufficient to meet the basic requirements of mDNS.

Q2.Step1.a

```
C:\Users\ASUS>tracert httpbin.org

通过最多 30 个跃点跟踪
到 httpbin.org [54.152.142.77] 的路由:

  1    2 ms    9 ms    2 ms    10.176.0.2
  2    3 ms    3 ms    2 ms    10.253.0.4
  3    8 ms    8 ms    5 ms    124.89.192.65
  4    *      *      *      请求超时。
  5    *      *      *      请求超时。
  6    *      *      *      请求超时。
  7    *      *      *      请求超时。
  8    *      *      47 ms   219.158.9.202
  9   33 ms    *      34 ms   219.158.9.62
 10    *      196 ms   *      219.158.16.94
 11   181 ms   177 ms   *      219.158.43.6
 12    *      *      *      请求超时。
 13    *      *      *      请求超时。
 14    *      *      *      请求超时。
 15    *      *      *      请求超时。
 16    *      *      *      请求超时。
 17    *      *      *      请求超时。
 18    *      *      *      请求超时。
 19    *      *      *      请求超时。
 20    *      *      *      请求超时。
 21    *      *      *      请求超时。
 22    *      *      *      请求超时。
 23    *      *      *      请求超时。
 24    *      *      *      请求超时。
 25    *      *      *      请求超时。
 26    *      *      *      请求超时。
 27    *      *      *      请求超时。
 28    *      *      *      请求超时。
 29    *      *      *      请求超时。
 30    *      *      *      请求超时。

跟踪完成。
```

- **Protocol and Layer:** The protocol used by traceroute is ICMP (Internet Control Message Protocol), which belongs to the Network Layer of the TCP/IP model.

- **Main Characteristics:** ICMP is mainly used to transmit control messages between IP hosts and routers, such as messages about whether the network is accessible, whether the host is reachable, and whether the route is available. It does not handle the transmission of user data but is used for diagnosing network connection status.

- **Working Principle:** Traceroute sends ICMP Echo request packets with different TTL (Time - to - Live) values. Initially, the TTL is set to 1. When the packet reaches the first router, the TTL decrements to 0, and the router returns an ICMP Time Exceeded message to the source host. Then, traceroute gradually increases the TTL value and repeatedly sends ICMP Echo requests until it reaches the destination host. The destination host will return an ICMP Echo Reply message. In this way, traceroute can record the routers along the path.

- **Use Case Examples:** Network administrators can use traceroute (based on ICMP) to troubleshoot network failures. For example, they can determine at which node along the network link from the local end to the target server there is a delay or packet loss problem, thus locating the network failure point.

Q2.Step1.b

1. Interpretation of Traceroute Output

- **Normal Hop Information:** In the traceroute output, the first few hops (such as hop 1 - 3) show Round-Trip Time (RTT), for example, the RTT of hop 1 is 2 ms, 9 ms, and 2 ms respectively. This indicates that the communication delay from the local host to these routers is relatively low and stable. These values reflect the time taken by data packets to pass through these nodes in the network.

- Abnormal Measurement Values

- **Request Timeout (***):** Starting from hop 4, multiple "Request Timeout" (represented by *) situations occurred. This means that at these hops, the ICMP Echo request data packets sent by traceroute did not receive the expected responses. Possible reasons include:

- Intermediate routers configured not to return ICMP timeout messages. Due to security policies and other reasons, they may discard these data packets without sending responses.
- Network links at these nodes failed, resulting in data packets being unable to be transmitted normally and returning responses.

- **Larger Delay Values:** At hop 9, there are delay values of 33 ms, *, and 34 ms. One value is missing (*), and the overall delay has increased compared to the previous hops. At hop 10, the delay value reaches 196 ms, which is significantly different from the previous hop delays. This large difference may be due to network congestion at this node or the limited processing capacity of the router, causing data packets to queue up and wait for processing, thereby increasing the delay.

31	2025-04-10 13:17:33.484814	10.176.48.13	10.2.1.5	NBNS	110 Refresh NB LAPTOP-9J5RKR31<20>
34	2025-04-10 13:17:34.600226	10.176.48.13	54.152.142.77	ICMP	106 Echo (ping) request id=0x0001, seq=91/23296, ttl=1 (no response found!)
35	2025-04-10 13:17:34.602931	10.176.0.2	10.176.48.13	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
36	2025-04-10 13:17:34.603534	10.176.48.13	54.152.142.77	ICMP	106 Echo (ping) request id=0x0001, seq=92/23552, ttl=1 (no response found!)
37	2025-04-10 13:17:34.612524	10.176.0.2	10.176.48.13	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
38	2025-04-10 13:17:34.615310	10.176.48.13	54.152.142.77	ICMP	106 Echo (ping) request id=0x0001, seq=93/23808, ttl=1 (no response found!)
39	2025-04-10 13:17:34.617774	10.176.0.2	10.176.48.13	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
40	2025-04-10 13:17:34.618803	10.176.48.13	10.178.2.128	DNS	83 Standard query 0x3571 PTR 2.0.176.10.in-addr.arpa

2.This diagram provides strong evidence:

- Regarding Timeouts of Requests

Records numbered 34, 36, and 38 in the diagram are ICMP Echo (ping) requests sent from the source address `10.176.48.13` to the destination address `54.152.142.77`, and it is clearly marked `(no response found!)`. This indicates that these requests did not receive the expected responses, corresponding to the "Request timed out" (*) situations in the traceroute output. It confirms that there were issues with packet transmission during these processes, such as potential network problems or policy restrictions of intermediate devices.

- Regarding ICMP Time - Exceeded Messages

Records numbered 35, 37, and 39 show that the source address `10.176.0.2` sent `Time - to - live exceeded (Time to live exceeded in transit)` messages to `10.176.48.13`, that is, ICMP time - exceeded messages. This indicates that when the sent ICMP Echo request packets passed through the router at `10.176.0.2`, the TTL value decremented to 0, which is consistent with the working principle of the traceroute tool that traces the route path by setting different TTL values. It further corroborates the reliability of the hop delay and other data in the traceroute output.

Q2.Step2.a

```
C:\Users\ASUS>tracert leetcode.cn

通过最多 30 个跃点跟踪
到 1zi859in9rs1950w.aliyunddos1010.com [203.107.63.81] 的路由:

  1      9 ms      18 ms      9 ms      172.19.127.254
  2      1 ms      1 ms      1 ms      172.20.253.6
  3      2 ms      2 ms      3 ms      169.254.3.5
  4      3 ms      3 ms      12 ms     115.233.67.225
  5      4 ms      *          2 ms      61.175.24.21
  6      *          *          *          请求超时。
  7      *          *          *          请求超时。
  8      *          *          *          请求超时。
  9      *          *          *          请求超时。
 10     29 ms     32 ms     28 ms     113.96.53.10
 11      *          *          *          请求超时。
 12      *          *          *          请求超时。
 13      *          *          *          请求超时。
 14      *          *          *          请求超时。
 15     25 ms     25 ms     25 ms     203.107.63.81

跟踪完成。
```

1. Traceroute output interpretation

- **Normal jump information:** The delay time is relatively low and stable for the first 4 jumps. For example, the delay time of jump 1 is 9 ms, 18 ms, and 9 ms respectively. The delay of jump 2 is 1 ms. This indicates that in the initial part of the network path, the packet transmission is smooth, the network equipment processing speed is fast, and the network condition is good.

- Abnormal measurement

- **Request timeout (**):** The request timeout condition begins at jump 5 and continues at jump 6-9 and jump 11-14. This means that at these hops, the ICMP Echo request packet sent by traceroute did not receive the expected response. Possible causes are:

- Security policies are configured on the intermediate router to prevent ICMP response packets from returning. For example, access control lists (ACLs) are configured to filter ICMP traffic.
- Network links fail at these nodes, preventing packets from being transmitted and responses received.

- **Delay fluctuation:** Jump 10 has a relatively high and volatile delay (29 ms, 32 ms, 28 ms), which is quite different from the lower delay of the previous normal jump. This may be due to congestion in the node's network, or the router's insufficient processing power, causing packets to queue up, thereby increasing the transmission delay.

- **Delay returns to normal:** The delay of hop 15 returns to a relatively stable and low level (both 25 ms), indicating that the network transmission returns to a relatively normal state after passing the previous faulty node.

No.	Time	Source IP	Destination IP	Protocol	Details
931	2025-04-11 01:11:47.388379	113.96.53.10	172.19.111.130	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
932	2025-04-11 01:11:47.389705	172.19.111.130	203.107.63.81	ICMP	106 Echo (ping) request id=0x0001, seq=305/12545, ttl=10 (no response found)
933	2025-04-11 01:11:47.422367	113.96.53.10	172.19.111.130	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
934	2025-04-11 01:11:47.423599	172.19.111.130	203.107.63.81	ICMP	106 Echo (ping) request id=0x0001, seq=306/12801, ttl=10 (no response found)
935	2025-04-11 01:11:47.451875	113.96.53.10	172.19.111.130	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)

2.This Wireshark packet capture diagram strongly supports the previous analysis of traceroute www.google.cn:

- About Request timeout

Records numbered 932 and 934 in the figure show that source IP address 172.19.111.130 sends ICMP Echo (ping) requests to target IP address 203.107.63.81 and is marked with "no response found!". Corresponding to the case of multiple hop requests time-out in the traceroute output, it is confirmed that the packets did not receive the expected response during these transmission processes, and further indicates that there is a problem in network transmission, which may be an intermediate device policy limitation or a link failure.

- About ICMP timeout messages

Records with serial numbers 931, 933, and 935 display source address 113.96.53.10 returning Time to live exceeded in transit (Time to live exceeded in transit) to 172.19.111.130. That is, ICMP timeout message. This is consistent with the working principle of the traceroute tool to trace the route path by setting different TTL values. It indicates that when the sent ICMP Echo request packet passes through the node 113.96.53.10, the TTL value decreases to 0. The reliability of the data such as jump delay in traceroute output is further proved, and the basis for analyzing the network transmission path is also provided.

Q2.Step2.b

1. Major differences

- Jump response

- traceroute to httpbin.org: A large number of "Request timeouts" (*) occur from hop 4 and continue to multiple hops.
- traceroute to leetcode.cn: "Request timeout" starts at hop 5, with a response at hop 10 in the middle, and then a request timeout occurs at multiple hops. In contrast, leetcode.cn has a partial successful jump response, unlike httpbin.org, which has almost no subsequent effective response after a timeout.

-Delay condition

- traceroute to httpbin.org: The delay fluctuates greatly. In the responsive jump, for example, the delay of the 10th jump reaches 196 ms, and the delay difference between different jumps is obvious.
- traceroute to leetcode.cn: The normal jump delay in the early stage is relatively stable and low, such as jump 1-4. Hop 10 latency is relatively high but the fluctuation range is relatively small compared to httpbin.org, and eventually hop 15 latency returns to stable.

-Ultimate reachability

- traceroute to httpbin.org: Although the trace is eventually shown to be complete, a large number of timeouts in between may indicate poor network connection quality.
- traceroute to leetcode.cn: The destination address (hop 15) is successfully reached, indicating that the destination address is reachable from the overall path.

2. Explanation of differences

- Network path and device configuration

httpbin.org: The network path may pass through a large number of network devices that impose strict ICMP restrictions, or the network link may have a large number of unstable areas, resulting in a large number of request timeout. For example, some network operators restrict ICMP requests for external access for security or traffic management purposes.

leetcode.cn: Some nodes on the network path may have ICMP restrictions, but the overall network configuration of routes for this domain name is relatively good, so that some nodes

can respond normally and finally reach the target. Perhaps its network has been optimized through the carrier network to ensure the connectivity of key nodes.

- Network congestion

httpbin.org: Network congestion can be severe, resulting in large latency fluctuations and many timeouts. For example, on some international network links, due to heavy traffic and limited link bandwidth, packets queue for a long time, and the transmission delay is increased or even lost.

leetcode.cn: The overall network congestion is relatively good. Although some nodes have problems, the overall connectivity is not seriously affected. It is possible that the server deployment and network architecture design took into account factors such as traffic balance.